

東京 2025 Linux Plumbers Conference

TOKYO, JAPAN / DECEMBER 11-13, 2025

Juan Yescas & Kalesh Singh

Google



HW/SW design recommendations for 16KB devices

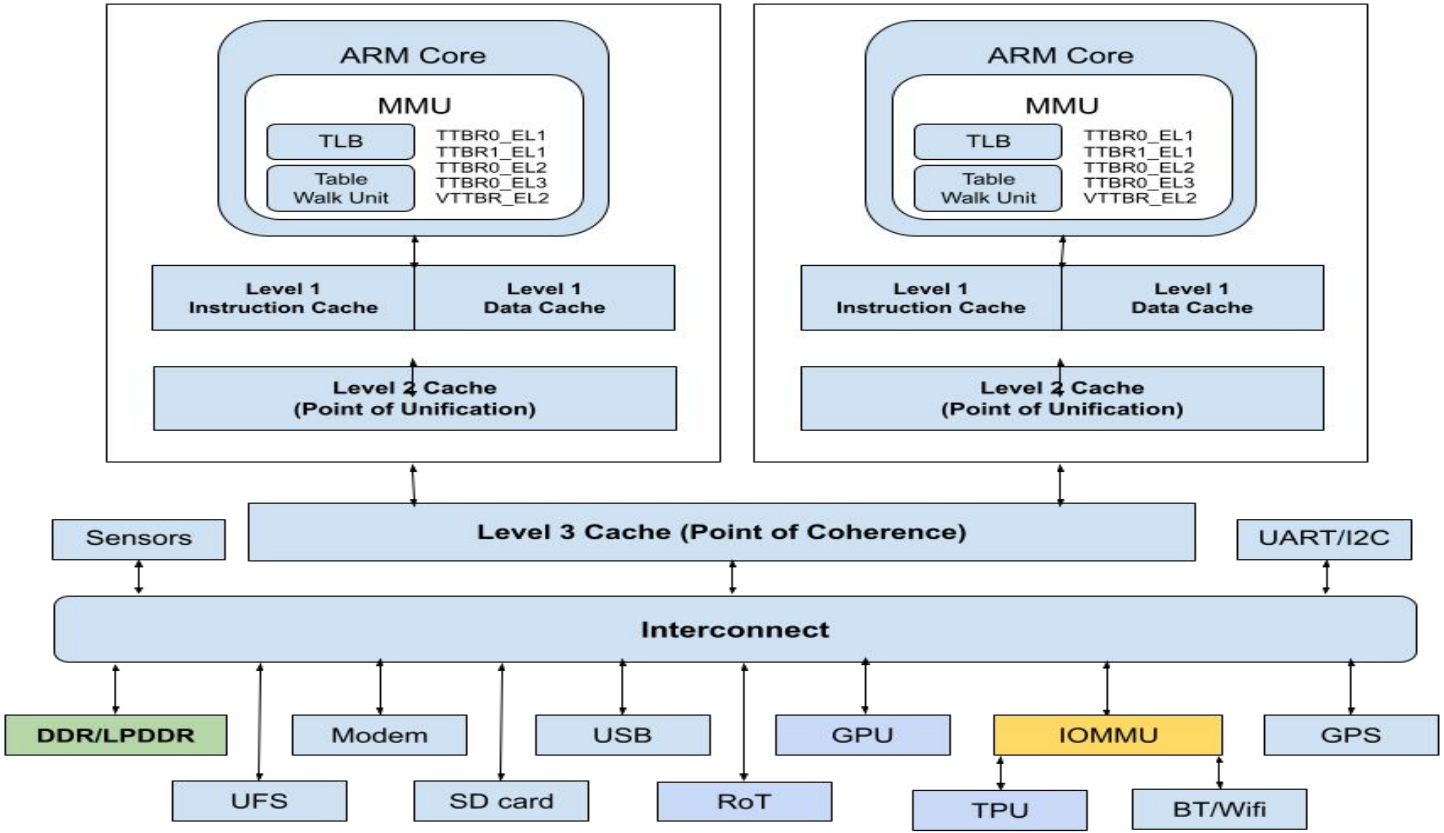
Juan Yescas & Kalesh Singh

Google



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

System on Chip



IOMMU/SysMMU HW

Recommendation:

- Use the same page size for the Linux kernel and the IOMMU/SysMMU

Benefits:

- The IOMMU/SysMMU cache will have less cache misses.
- More memory can be addressed by the IOMMU/SysMMU

Trade Offs:

- The address space for the IO devices is larger, this means that some microprocessors might not have enough VA memory.
- This could affect old 32-bit microcontrollers in the SoC

Compression/Decompression HW

Recommendation:

- Use block sizes equal to the page size for the compression/decompression HW.

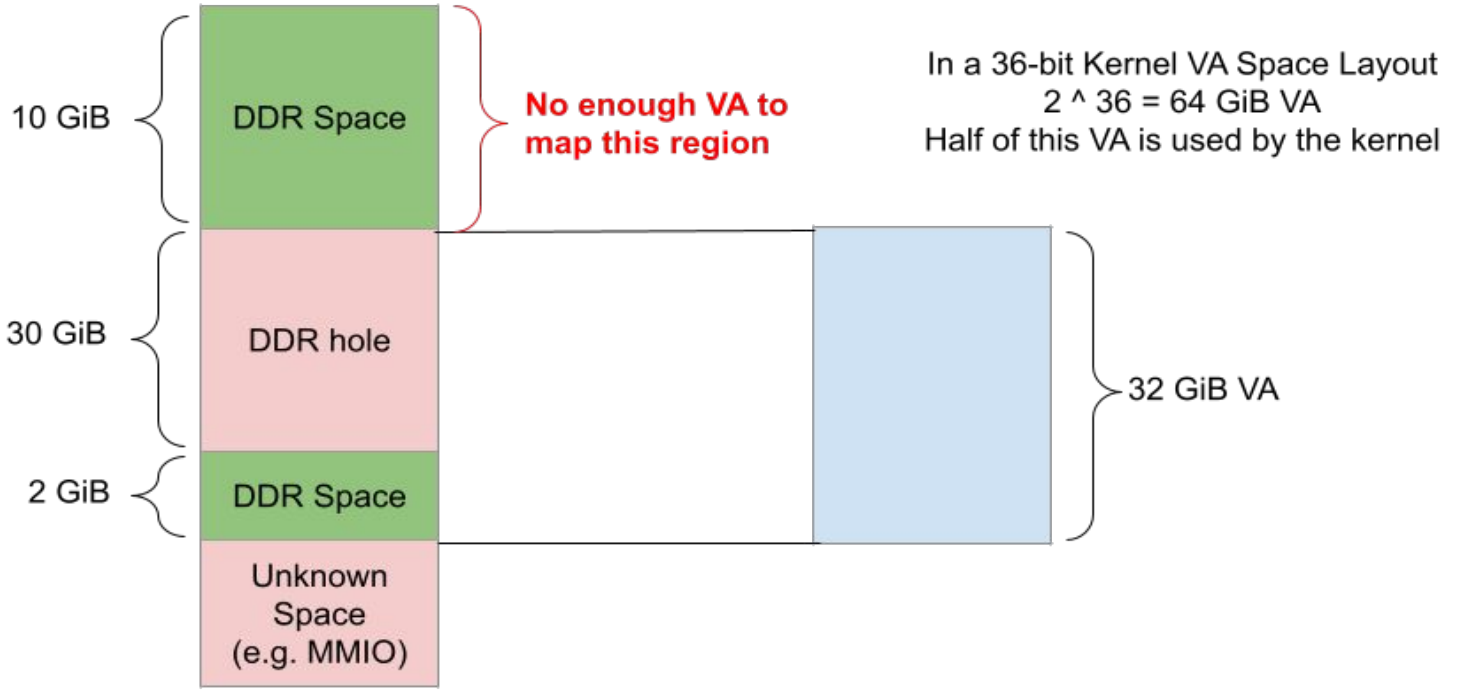
Benefits:

- Compression rates are better in large page sizes.
- In 4 KB, the compression ratio is 2.5 to 1.
- In 16 KB, the compression ratio is 3.3 to 1.

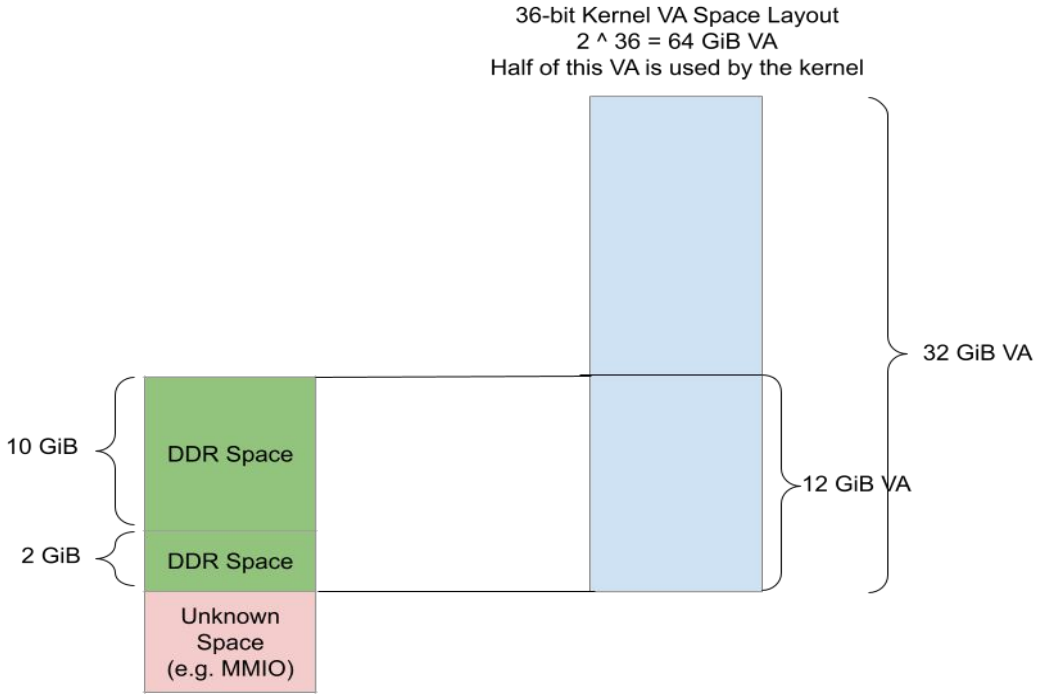
Trade Offs:

- For small files, there could be more fragmentation in large page sizes.

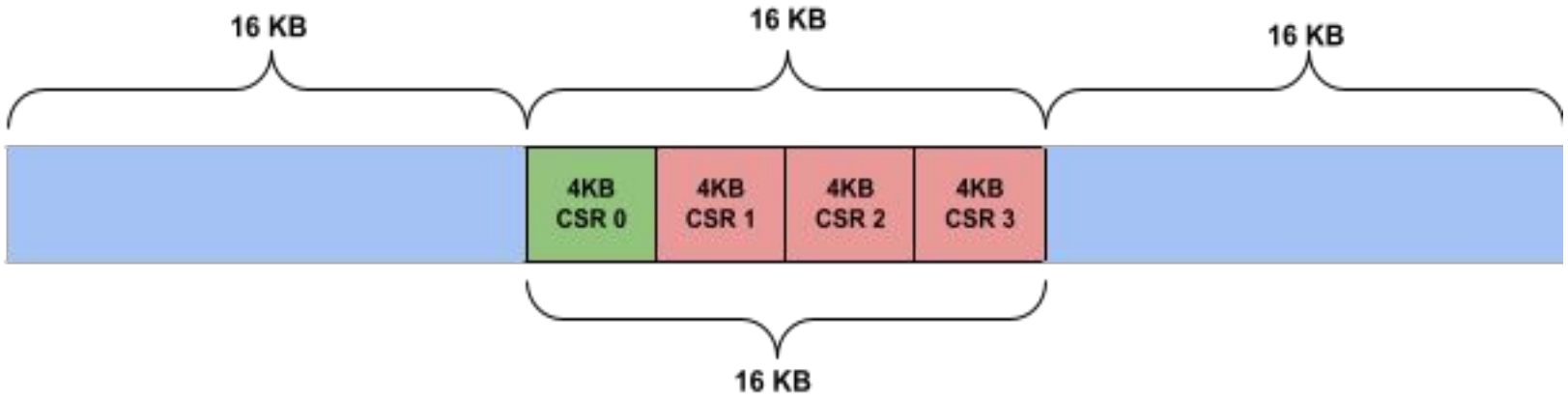
Don’t use memory holes between DDR



DDRs set up without holes

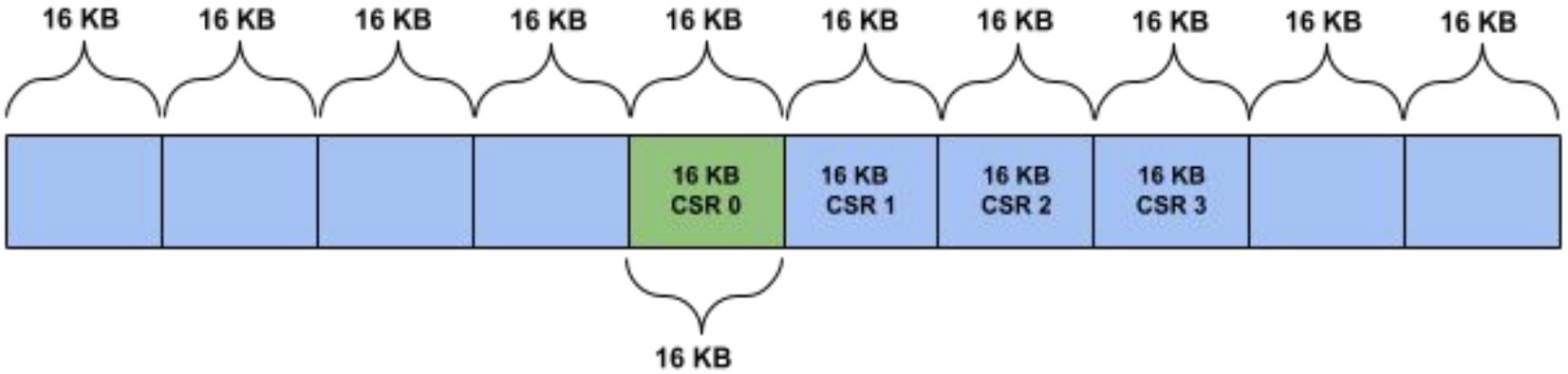


Aligned CSRs with 16 KB alignment



Userspace or driver only wants to map the CSR 0, however, it gets access to CSR 1, CSR 2, CSR3.

Aligned CSRs with 16 KB alignment



Userspace or driver only wants to map the CSR 0

Aligned CSRs with 16 KB alignment

Recommendation:

- Aligned CSRs with 16 KB alignment

Benefits:

- It will prevent the user space from having illegal accesses to other CSRs.

Trade Offs:

- The address space for CSRs that are 16KB aligned will be larger, this means that some microprocessors might not have enough VA memory to map all the registers or parts of the RAM.

Factory Partitions formatting

Recommendation:

- Format factory partitions with **ext4** filesystem **AND 4kb block size**.

Benefits:

- It will allow the devices that boots with 4kb or 16kb page sizes to read the factory partitions.

Trade Offs:

- On 16kb kernels, loading files smaller than 16 KB in size could cause memory fragmentation.

CMA memory alignment issues

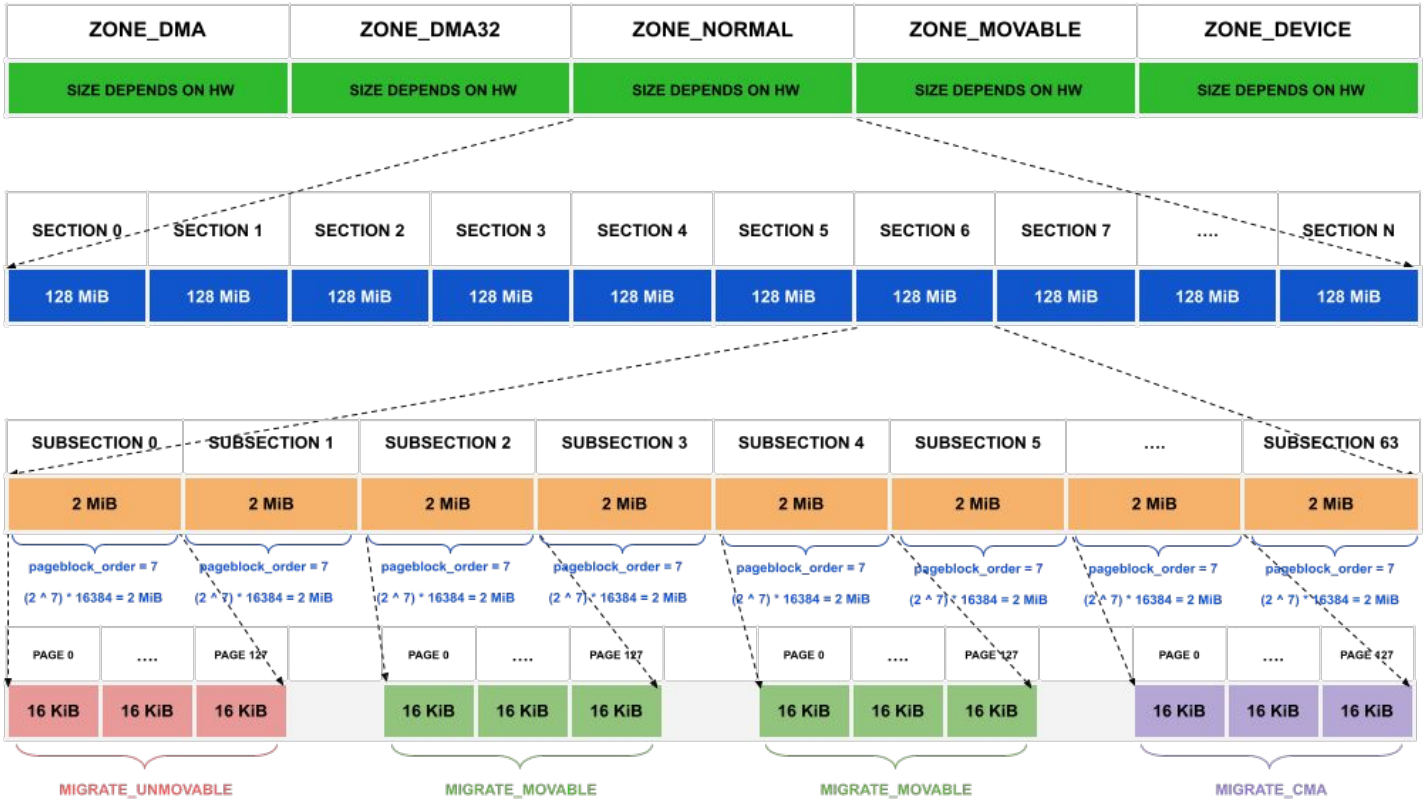
```
# 4KiB kernel reservations

reserved-memory {
    ...
    cma_test_reserve: cma_test_reserve {
        compatible = "shared-dma-pool";
        size = <0x0 0x200000>; /* 2 MiB */
        ...
    };
};
```

```
# 16 KiB kernel reservations

reserved-memory {
    ...
    cma_test_reserve: cma_test_reserve {
        compatible = "shared-dma-pool";
        size = <0x0 0x2000000>; /* 32 MiB */
        ...
    };
};
```

CONFIG_PAGE_BLOCK_MAX_ORDER=7 in 6.17 kernel



CONFIG_PAGE_BLOCK_MAX_ORDER=7 in 6.17 kernel

Recommendation:

- Set the **CONFIG_PAGE_BLOCK_MAX_ORDER=7** **in** **6.17+ Linux kernels** to have the same alignment requirements that 4 KB kernels.

Benefits:

- The CMA memory reservations won't increase in 16 KB page size kernels.

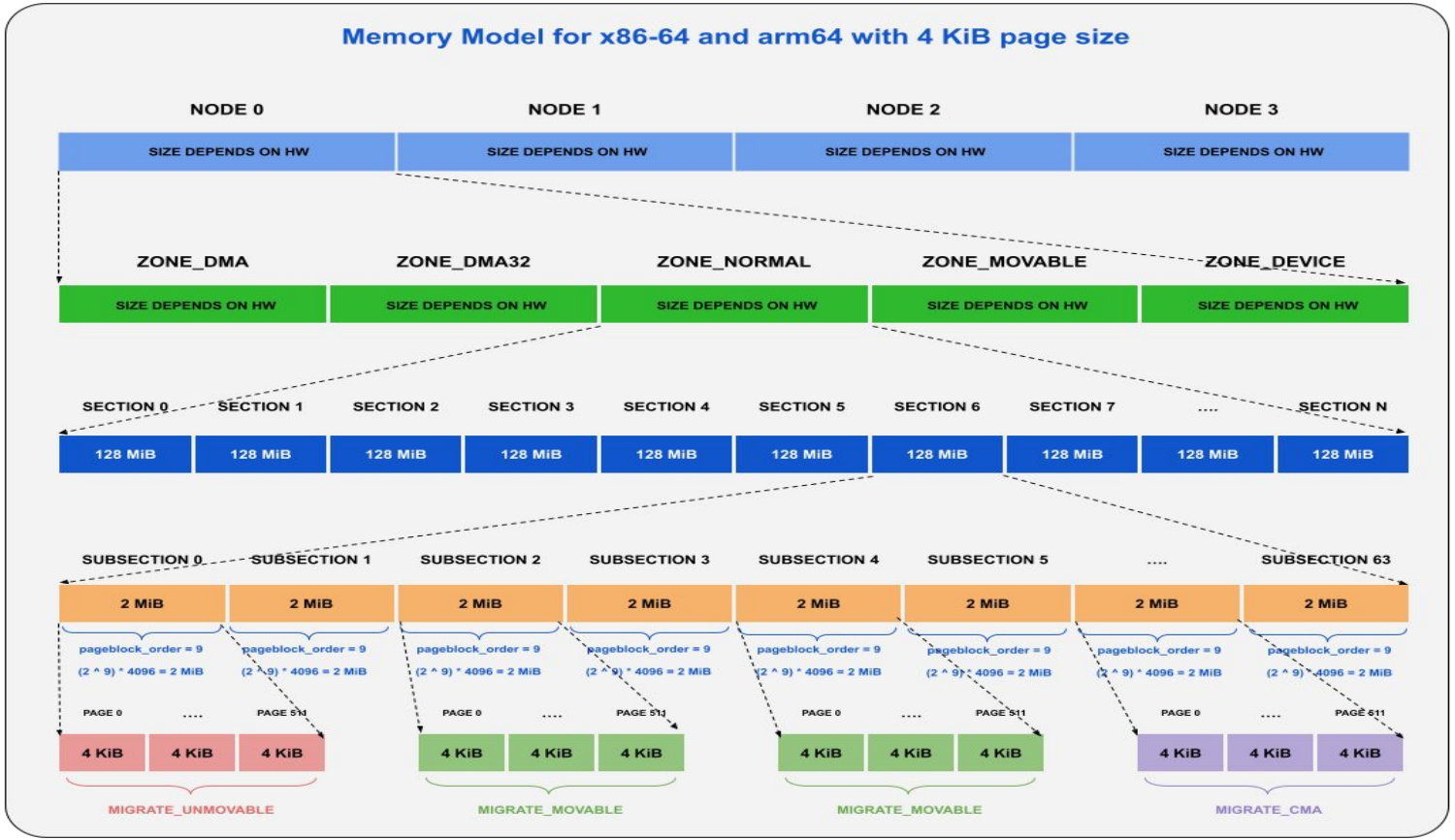
Trade Offs:

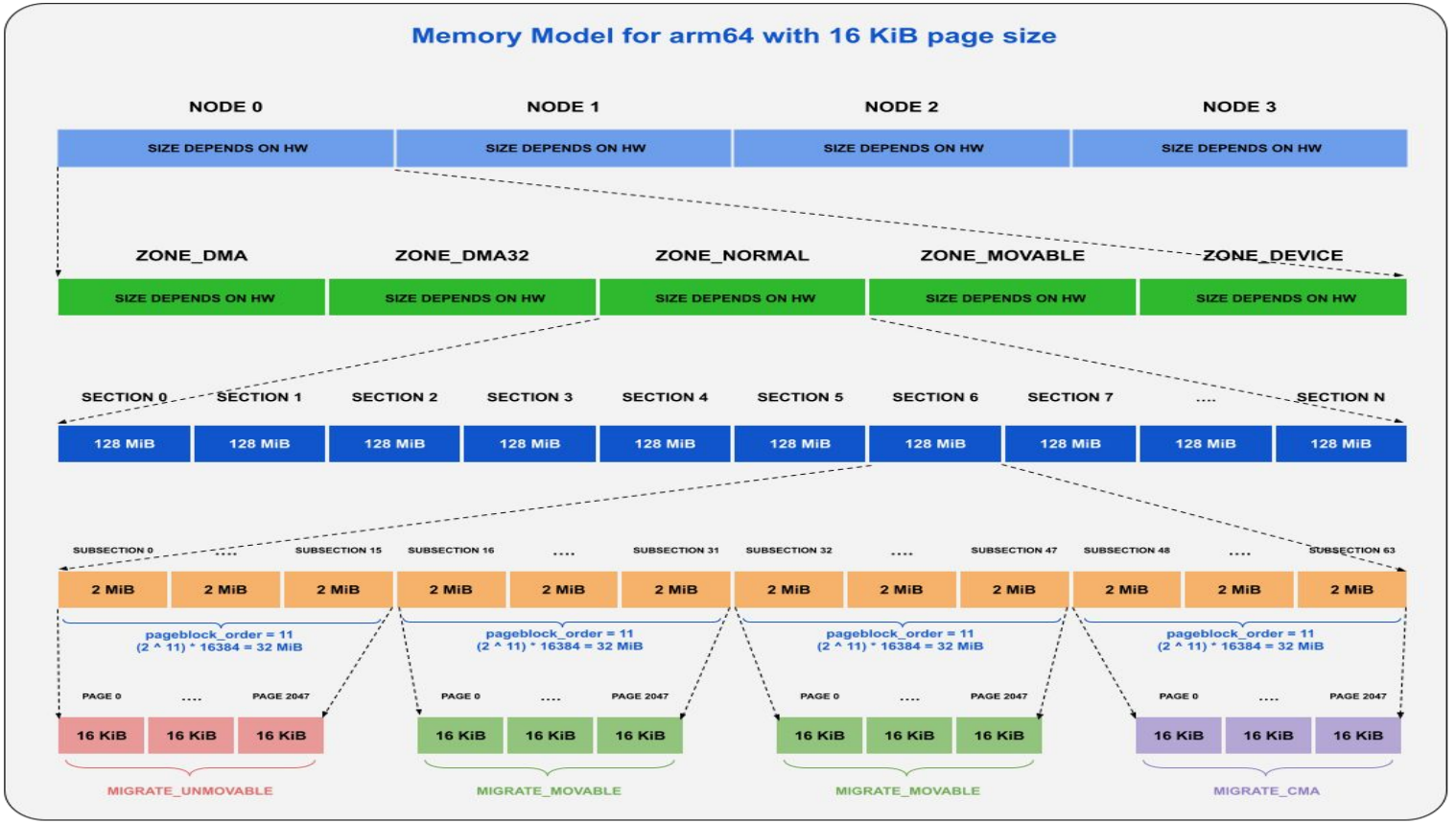
- Reducing pageblock order can negatively impact THP generation success rate. If your workloads use THP heavily, please use this option with caution.

Questions

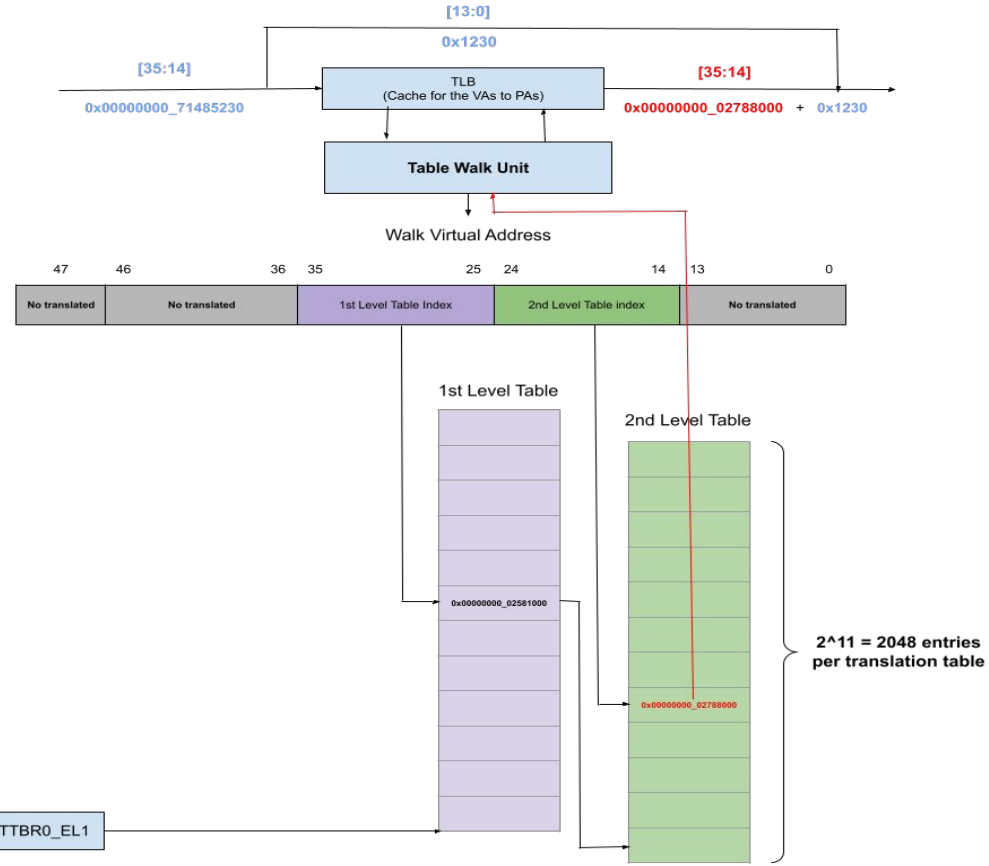
Thanks

Appendix


Memory Model for x86-64 and arm64 with 4 KiB page size
NODE 0
NODE 1
NODE 2
NODE 3
SIZE DEPENDS ON HW
ZONE_DMA
ZONE_DMA32
ZONE_NORMAL
ZONE_MOVABLE
ZONE_DEVICE
SECTION 0
SECTION 1
SECTION 2
SECTION 3
SECTION 4
SECTION 5
SECTION 6
SECTION 7
....
SECTION N
128 MiB
SUBSECTION 0
SUBSECTION 1
SUBSECTION 2
SUBSECTION 3
SUBSECTION 4
SUBSECTION 5
....
SUBSECTION 63
2 MiB
pageblock_order = 9
(2 ^ 9) * 4096 = 2 MiB
PAGE 0
....
PAGE 511
4 KiB
MIGRATE_UNMOVABLE
MIGRATE_MOVABLE
MIGRATE_MOVABLE
MIGRATE_CMA


Memory Model for arm64 with 16 KiB page size
NODE 0
NODE 1
NODE 2
NODE 3
SIZE DEPENDS ON HW
SIZE DEPENDS ON HW
SIZE DEPENDS ON HW
SIZE DEPENDS ON HW
ZONE_DMA
ZONE_DMA32
ZONE_NORMAL
ZONE_MOVABLE
ZONE_DEVICE
SIZE DEPENDS ON HW
SIZE DEPENDS ON HW
SIZE DEPENDS ON HW
SIZE DEPENDS ON HW
SIZE DEPENDS ON HW
SECTION 0
SECTION 1
SECTION 2
SECTION 3
SECTION 4
SECTION 5
SECTION 6
SECTION 7
....
SECTION N
128 MiB
128 MiB
128 MiB
128 MiB
128 MiB
128 MiB
128 MiB
128 MiB
128 MiB
128 MiB
SUBSECTION 0
....
SUBSECTION 15
SUBSECTION 16
....
SUBSECTION 31
SUBSECTION 32
....
SUBSECTION 47
SUBSECTION 48
....
SUBSECTION 63
2 MiB
pageblock_order = 11
(2 ^ 11) * 16384 = 32 MiB
PAGE 0
....
PAGE 2047
16 KiB
MIGRATE_UNMOVABLE
MIGRATE_MOVABLE
MIGRATE_MOVABLE
MIGRATE_CMA

Page Table Walks - 16k Granule - 36-bits VA



Page Table Walks - 16k Granule - 47-bits VA

